

# NEXA v1

## Architecture & Product Specification

*Internal Social + Communication Platform for Organizations*

Living document • Implementation blueprint • v1.0

# 1. Product Definition

NEXA Workplace is a private social and communication platform for organizations. Its purpose is to separate workplace communication from employees' personal messaging spaces while creating a reusable identity, organization, communication and notification layer for future SaaS products.

## 1.1 Product vision

NEXA Workplace  
= private company social feed  
+ direct/group/channel chat  
+ organization & identity  
+ notifications  
+ files  
+ administration  
+ platform APIs/events for future SaaS integration

## 1.2 Positioning

Working product statement:

Your company's private digital workspace.

Alternative:

One place for your people, conversations and company life.

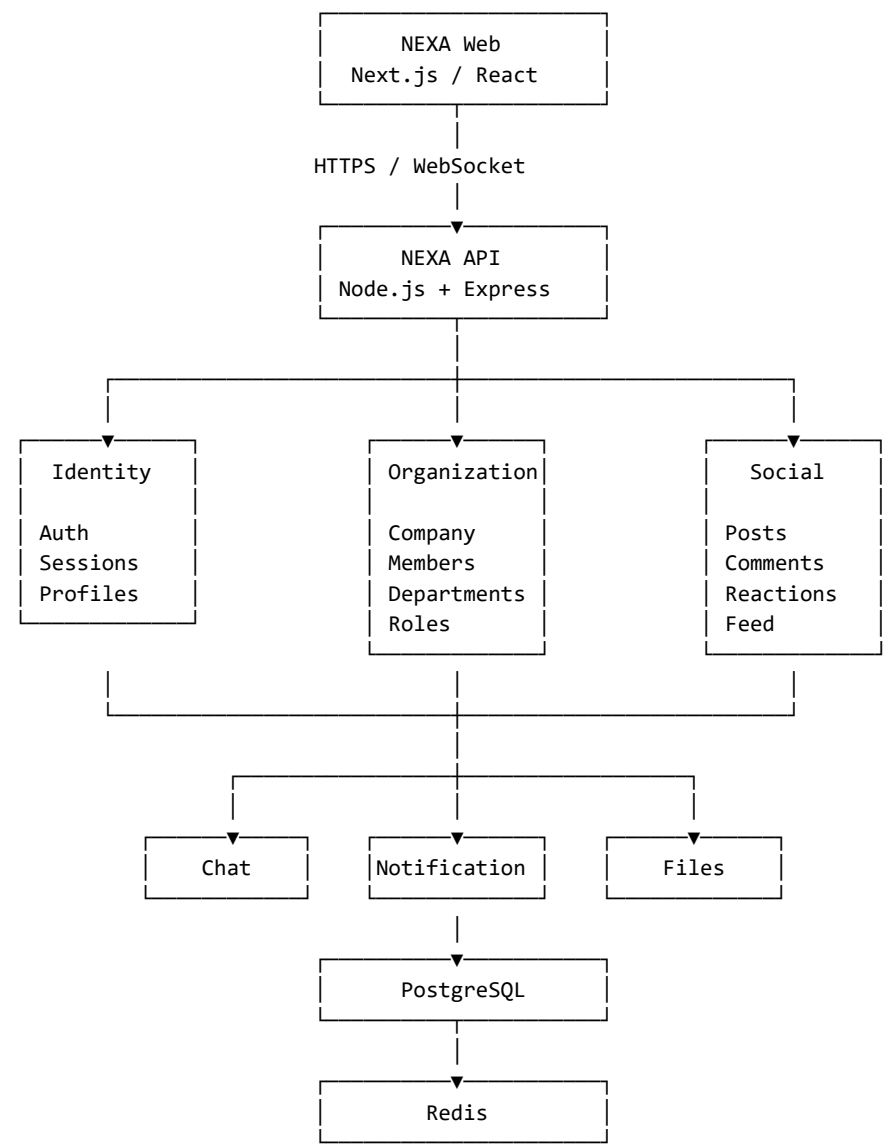
## 1.3 Core problem

- Work communication is often mixed with employees' personal communication.
- Companies need a private place for company-wide updates, department communication and chat.
- Future SaaS products need reusable identity, organization, notification and communication capabilities.

# 2. Scope & Product Principles

| Principle                  | Decision                                                                           |
|----------------------------|------------------------------------------------------------------------------------|
| Multi-tenant first         | Every business resource is scoped to an Organization.                              |
| Modular monolith           | One deployable backend with strong domain boundaries; no microservices in v1.      |
| API-first                  | Web and future clients consume HTTP/WebSocket APIs.                                |
| Identity ≠ Organization    | A User is an identity; company membership is modeled separately.                   |
| PostgreSQL source of truth | Redis is for cache, presence, rate limiting and pub/sub/temporary state.           |
| Event-oriented modules     | Business modules publish domain events rather than hard-coding cross-module calls. |
| Design for extraction      | Boundaries should allow future service extraction without forcing it now.          |
| Avoid over-engineering     | Implement only what creates product value in v1.                                   |

### 3. Architecture Overview



### 4. Domain Modules

| Module        | Owns                                                                          | Does not own                 |
|---------------|-------------------------------------------------------------------------------|------------------------------|
| Identity      | Users, authentication, sessions, refresh tokens, profiles                     | Departments, company roles   |
| Organization  | Organizations, memberships, departments, roles, permissions                   | Passwords, posts             |
| Social        | Posts, comments, reactions, feed, mentions, bookmarks, attachments references | Authentication               |
| Communication | Conversations, channels, members, messages, presence, read state              | User credentials             |
| Notification  | In-app notifications and notification delivery rules                          | Business entities themselves |
| File          | File metadata, uploads, attachment                                            | Post/message business logic  |

|                |                                                     |                      |
|----------------|-----------------------------------------------------|----------------------|
|                | references                                          |                      |
| Search         | Search across permitted users/posts/channels        | Primary data storage |
| Administration | Moderation, audit logs, organization administration | Core domain entities |

## 5. Dependency Rules

```

Identity
↓
Organization
↓
Social / Communication
↓
Notification
↓
Delivery integrations

```

Shared infrastructure is available to all modules, but modules must not bypass another module's public application/domain boundary.

**NOTE** Do not allow circular dependencies such as Chat → Social → Notification → Chat. Use domain events for cross-module reactions.

## 6. Identity Module

### 6.1 Responsibilities

- Registration
- Login
- JWT access tokens
- Refresh tokens
- Logout/revocation
- Password change
- User profile
- Session management

### 6.2 User model

```

users
-----
id UUID PK
email UNIQUE
username UNIQUE
display_name
password_hash
avatar_url NULL
bio NULL
status
created_at
updated_at
last_login_at NULL

```

## 6.3 Authentication flow

```
Login
↓
Validate credentials
↓
Issue short-lived access token
+
Issue refresh token
↓
Client uses access token
↓
Access token expires
↓
POST /auth/refresh
↓
Issue new access token
```

Recommended starting policy: access token ~15 minutes; refresh token ~30 days. Make expiry configurable.

## 6.4 Token storage

Store a hash of refresh tokens in the database rather than the raw refresh token.

# 7. Organization Module

## 7.1 Organization

```
organizations
-----
id UUID PK
name
slug UNIQUE
logo_url NULL
timezone
settings JSONB
created_at
updated_at
```

## 7.2 Membership

```
organization_members
-----
id UUID PK
organization_id FK
user_id FK
role_id FK
status
joined_at
created_at
updated_at

UNIQUE(organization_id, user_id)
```

## 7.3 Departments

departments

-----

id UUID PK  
organization\_id FK  
name  
slug  
description NULL  
created\_at  
updated\_at

department\_members

-----

department\_id FK  
user\_id FK  
PRIMARY KEY(department\_id, user\_id)

## 7.4 RBAC

roles  
permissions  
role\_permissions

Example roles:

OWNER  
ADMIN  
MANAGER  
MEMBER

Permissions should be explicit, e.g. member.invite, member.remove, post.moderate, channel.create, message.delete, audit.read.

# 8. Social Module

## 8.1 Post

posts

-----

id UUID PK  
organization\_id FK  
author\_id FK  
content  
type  
visibility  
created\_at  
updated\_at  
deleted\_at NULL

type:

GENERAL | ANNOUNCEMENT | EVENT | POLL

visibility:

ORGANIZATION | DEPARTMENT | CHANNEL

## 8.2 Comment

comments

```
-----  
id UUID PK  
post_id FK  
author_id FK  
parent_id NULL  
content  
created_at  
updated_at  
deleted_at NULL
```

parent\_id enables nested replies.

## 8.3 Reactions

reactions

```
-----  
id UUID PK  
user_id FK  
post_id FK  
type  
created_at
```

UNIQUE(user\_id, post\_id)

Initial reaction types: LIKE, LOVE, HAHA, CELEBRATE, SAD.

## 8.4 Feed

GET /api/v1/feed

Feed sources in v1:

- organization posts
- department posts
- announcements
- followed-user posts (if follow is implemented)

Use cursor pagination:

```
{  
  "data": [],  
  "pagination": {  
    "next_cursor": "...",  
    "has_next": true  
  }  
}
```

# 9. Communication Module

## 9.1 Conversation model

conversations

```
-----  
id UUID PK  
organization_id FK  
type
```

```
name NULL
description NULL
avatar_url NULL
created_by FK
created_at
updated_at
last_message_at NULL

type:
DIRECT | GROUP | CHANNEL
```

## 9.2 Members

```
conversation_members
-----
conversation_id FK
user_id FK
role
joined_at
last_read_message_id NULL
muted_until NULL
```

## 9.3 Messages

```
messages
-----
id UUID PK
conversation_id FK
sender_id FK
content
type
reply_to_id NULL
created_at
updated_at
deleted_at NULL

type:
TEXT | IMAGE | FILE | SYSTEM
```

## 9.4 Channel strategy

A channel is a Conversation with type=CHANNEL. Do not create a duplicate channel data model unless future requirements justify it.

# 10. Realtime Architecture

REST:

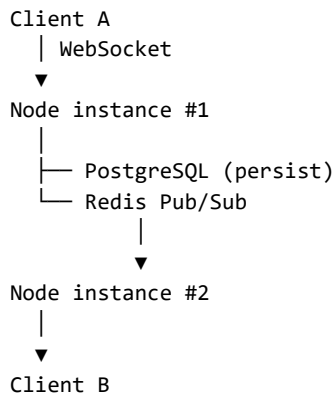
- conversation discovery
- message history
- durable message creation if desired
- settings/admin operations

WebSocket:

- message.created
- message.updated
- message.deleted



```
typing.started
typing.stopped
presence.updated
message.read
reaction.created
notification.created
```



Redis is especially useful when multiple API instances exist. Presence can be stored as short-lived Redis keys with TTL.

## 11. Notification Module

```
notifications
-----
id UUID PK
organization_id FK
recipient_id FK
actor_id NULL
type
entity_type
entity_id
metadata JSONB
read_at NULL
created_at
```

### 11.1 Domain events

```
PostLiked
CommentCreated
UserMentioned
MessageCreated
AnnouncementPublished
UserFollowed
```

Events are consumed by Notification without tightly coupling the producer to notification delivery.

## 12. File Module

```
Client
↓
```

```
POST /uploads
↓
File service
↓
Object storage
↓
CDN/object URL
↓
file record in PostgreSQL
```

files

-----

id UUID PK  
organization\_id FK  
uploaded\_by FK  
storage\_key  
filename  
mime\_type  
size  
url  
created\_at

post\_attachments  
message\_attachments

Do not store large binary files in PostgreSQL.

## 13. Search Module

v1: PostgreSQL full-text search. Search permitted Users, Posts and Channels.

GET /api/v1/search?q=react

Only introduce OpenSearch/Elasticsearch when scale or search requirements justify it.

## 14. Administration & Audit

audit\_logs

-----

id UUID PK  
organization\_id FK  
actor\_id FK  
action  
entity\_type  
entity\_id  
metadata JSONB  
ip\_address NULL  
user\_agent NULL  
created\_at

- Member management
- Departments
- Roles and permissions
- Moderation

- Organization settings
- Audit log viewing

## 15. Multi-Tenancy Rules

**NOTE** Every tenant-owned query must be scoped by `organization_id`. Knowing an entity UUID must never be enough to access another organization's data.

BAD:

```
SELECT * FROM posts WHERE id = $1;
```

GOOD:

```
SELECT * FROM posts
WHERE id = $1
AND organization_id = $2;
```

JWT

↓

user\_id

↓

organization membership

↓

organization\_id

↓

permission / ownership check

↓

query

Never trust `organization_id` supplied by the client when it can be derived from authenticated membership/context.

## 16. API Specification

### 16.1 Base

/api/v1

### 16.2 Auth & users

POST /auth/register

POST /auth/login

POST /auth/refresh

POST /auth/logout

POST /auth/change-password

GET /users/me

PUT /users/me

GET /users/:id

### 16.3 Organizations

POST /organizations

GET /organizations/:id

PUT /organizations/:id

GET /organizations/:id/members  
POST /organizations/:id/invitations

GET /organizations/:id/departments  
POST /organizations/:id/departments

## 16.4 Social

GET /feed

GET /posts/:id  
POST /posts  
PUT /posts/:id  
DELETE /posts/:id

GET /posts/:id/comments  
POST /posts/:id/comments  
DELETE /comments/:id

POST /posts/:id/reactions  
DELETE /posts/:id/reactions

## 16.5 Communication

GET /conversations  
POST /conversations

GET /conversations/:id  
GET /conversations/:id/messages  
POST /conversations/:id/messages

POST /conversations/:id/members  
DELETE /conversations/:id/members/:userId

## 16.6 Notifications & search

GET /notifications  
PATCH /notifications/:id/read  
POST /notifications/read-all

GET /search?q=

## 16.7 Response contract

Success:

```
{  
  "success": true,  
  "data": {}  
}
```

Error:

```
{  
  "success": false,  
  "error": {  
    "code": "POST_NOT_FOUND",  
    "message": "Post not found"  
  }  
}
```

```
  },  
  "request_id": "req_123"  
}
```

Use consistent HTTP status codes: 400 validation, 401 authentication, 403 authorization, 404 not found, 409 conflict, 429 rate limit, 500 unexpected error.

## 17. Security Architecture

```
HTTP Request  
↓  
Security headers  
↓  
CORS  
↓  
Rate limit  
↓  
Request ID  
↓  
JWT middleware  
↓  
Organization context  
↓  
Permission check  
↓  
Validation  
↓  
Controller  
↓  
Service  
↓  
Repository
```

- bcrypt password hashing.
- Short-lived access tokens and revocable refresh tokens.
- Input validation using a TypeScript-friendly schema library such as Zod.
- Security headers (Helmet or equivalent).
- Strict CORS configuration.
- Rate limiting on authentication and abuse-prone endpoints.
- Ownership checks for post/comment mutation.
- Tenant isolation on every organization-scoped query.

## 18. Backend Code Architecture

apps/api/src/

```
├── modules/  
│   ├── identity/  
│   │   ├── domain/  
│   │   ├── application/  
│   │   ├── infrastructure/  
│   │   └── presentation/  
│   ├── organization/  
│   └── social/
```

```

├── communication/
├── notification/
├── file/
├── search/
├── administration/
├── infrastructure/
│   ├── database/
│   ├── redis/
│   ├── storage/
│   ├── websocket/
│   └── logger/
├── shared/
│   ├── errors/
│   ├── events/
│   ├── types/
│   └── utils/
├── app.ts
└── server.ts

```

Recommended request flow:

Route → Validation → Controller → Application Service/Use Case → Repository → Database

**NOTE** Controllers should remain thin. Business rules belong in application/domain layers, not in route handlers.

## 19. Monorepo / Repository Layout

```

nexa/
├── apps/
│   ├── api/
│   └── web/
├── packages/
│   ├── types/
│   ├── config/
│   └── api-client/
├── infra/
│   ├── docker/
│   └── migrations/
├── docs/
│   ├── architecture/
│   ├── api/
│   ├── database/
│   └── adr/
├── docker-compose.yml
├── package.json
└── README.md

```

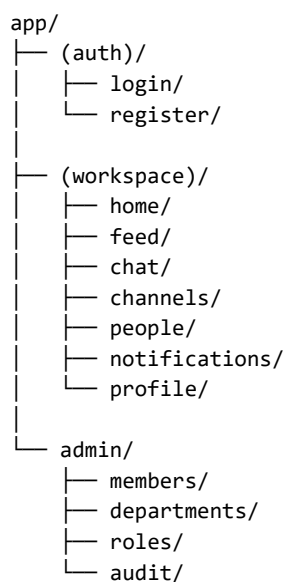
Suggested stack:

| Layer    | Technology        |
|----------|-------------------|
| Language | TypeScript        |
| Backend  | Node.js + Express |
| Database | PostgreSQL        |

|                |                             |
|----------------|-----------------------------|
| ORM            | Prisma or Drizzle           |
| Realtime       | WebSocket / Socket.IO or ws |
| Cache / PubSub | Redis                       |
| Web            | Next.js + React             |
| Server state   | TanStack Query              |
| Local UI state | React state / Zustand       |
| Validation     | Zod                         |
| API docs       | OpenAPI / Swagger           |
| Containers     | Docker / Docker Compose     |
| CI/CD          | GitHub Actions              |

## 20. Frontend Information Architecture

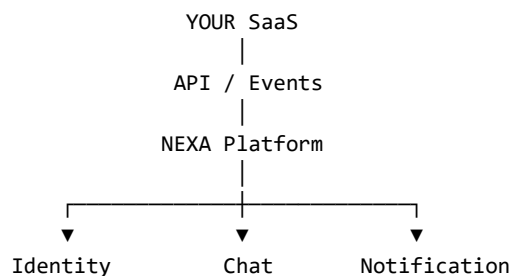
apps/web/



Core employee experience: Home/Feed, Messages, Channels, Activity/Notifications, People, Profile.

## 21. SaaS Integration Strategy

NEXA must expose capabilities through APIs/events rather than requiring future SaaS products to share its database.



### 21.1 Event envelope

```

{
  "id": "evt_123",

```

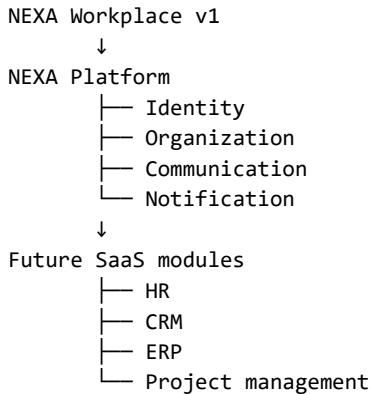
```

"type": "task.assigned",
"organization_id": "org_123",
"actor_id": "user_123",
"subject_id": "task_456",
"timestamp": "2026-09-12T08:00:00Z",
"metadata": {}
}

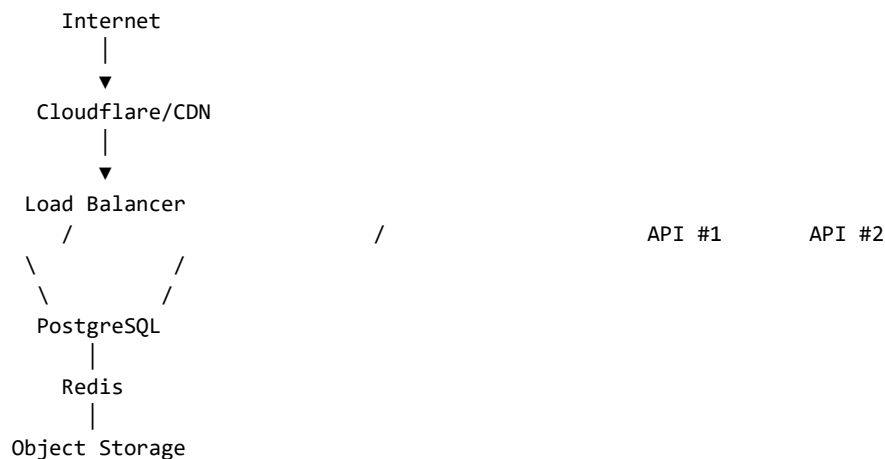
```

Example future integration: SaaS emits task.assigned → NEXA creates an in-app notification → user clicks through to the SaaS task URL.

## 21.2 Long-term platform decomposition



## 22. Deployment Architecture



Development should be reproducible with Docker Compose. Production topology can start with one API instance and scale horizontally when needed.

## 23. Observability

- Structured application logs.
- Request IDs propagated through requests and events.
- Health endpoint: GET /health.
- Readiness endpoint: GET /ready.



- Error tracking can be added with Sentry or equivalent.
- Metrics/tracing (OpenTelemetry/Prometheus/Grafana) can be added as scale requires.

## 24. Testing Strategy

| Level       | Focus                                                                             |
|-------------|-----------------------------------------------------------------------------------|
| Unit        | Domain rules, authorization, services/use cases                                   |
| Integration | API + PostgreSQL/Redis; repositories and critical workflows                       |
| E2E         | Register → login → organization → post → comment → reaction → chat → notification |

Minimum high-value authorization tests:

- Post owner can update/delete.
- Non-owner receives 403.
- Comment owner can delete.
- Cross-organization resource access is denied.
- Admin-only actions reject normal members.

## 25. Development Roadmap

### Phase 1 — Foundation

- ☐ TypeScript
- ☐ Express
- ☐ PostgreSQL
- ☐ ORM
- ☐ Docker
- ☐ Config
- ☐ Error system
- ☐ Logger
- ☐ OpenAPI

### Phase 2 — Identity

- ☐ Register
- ☐ Login
- ☐ JWT
- ☐ Refresh token
- ☐ Logout
- ☐ Profile

### Phase 3 — Organization

- ☐ Create organization
- ☐ Membership
- ☐ Invitation
- ☐ Departments

- ☐ RBAC

## Phase 4 — Social MVP

- ☐ Post
- ☐ Comment
- ☐ Reaction
- ☐ Feed
- ☐ Attachment
- ☐ Announcement

## Phase 5 — Communication

- ☐ Direct chat
- ☐ Group chat
- ☐ Channels
- ☐ Messages
- ☐ WebSocket
- ☐ Presence
- ☐ Read receipt

## Phase 6 — Platform

- ☐ Notifications
- ☐ Search
- ☐ Audit logs
- ☐ Admin

## Phase 7 — Web

- ☐ Workspace
- ☐ Feed
- ☐ Chat
- ☐ Profile
- ☐ Admin

## Phase 8 — Production

- ☐ Tests
- ☐ CI/CD
- ☐ Security review
- ☐ Monitoring
- ☐ Deployment

# 26. MVP Acceptance Criteria

- ☐ A user can register and log in securely.
- ☐ A user can belong to an organization through membership.
- ☐ Organization roles/permissions control privileged operations.
- ☐ Users can create, read, update and delete their own posts.

- ☐ Users can comment and react to posts.
- ☐ Feed is tenant-scoped and paginated.
- ☐ Users can create direct/group/channel conversations.
- ☐ Messages are persisted and delivered realtime.
- ☐ Presence/read state works for chat.
- ☐ Notifications are generated from relevant domain events.
- ☐ Files are stored outside PostgreSQL with metadata in PostgreSQL.
- ☐ Admins can manage members and inspect audit logs.
- ☐ Cross-tenant access is rejected.
- ☐ API documentation is available through OpenAPI.
- ☐ Project can run locally using Docker Compose.

## 27. Non-Goals for v1

- No microservices.
- No Kafka unless a real scaling requirement appears.
- No Elasticsearch/OpenSearch unless PostgreSQL search becomes insufficient.
- No video calls.
- No complex project-management suite.
- No enterprise SSO/SAML unless a target customer requires it.
- No algorithmic recommendation engine.
- No attempt to reproduce Facebook/Slack/Teams feature-for-feature.

## 28. Architecture Decision Records (ADR) to Maintain

| ADR     | Decision                                           |
|---------|----------------------------------------------------|
| ADR-001 | Use modular monolith for v1.                       |
| ADR-002 | Use PostgreSQL as primary source of truth.         |
| ADR-003 | Use multi-tenant organization/membership model.    |
| ADR-004 | Use short-lived JWT access + refresh tokens.       |
| ADR-005 | Use WebSocket for realtime communication.          |
| ADR-006 | Use Redis for presence/cache/pubsub/rate limiting. |
| ADR-007 | Use domain events for cross-module reactions.      |
| ADR-008 | Use API versioning under /api/v1.                  |

Every significant architecture change should create or update an ADR instead of silently changing the design.

## 29. Project Tracking Board

| Epic          | Status | Notes                              |
|---------------|--------|------------------------------------|
| Foundation    | TODO   | Project setup, Docker, DB, config  |
| Identity      | TODO   | Auth, sessions, profile            |
| Organization  | TODO   | Tenant, members, departments, RBAC |
| Social        | TODO   | Posts, comments, reactions, feed   |
| Communication | TODO   | Conversations, messages, WebSocket |
| Notification  | TODO   | Events + in-app notifications      |

|                |      |                               |
|----------------|------|-------------------------------|
| Files          | TODO | Object storage + attachments  |
| Search         | TODO | PostgreSQL FTS                |
| Administration | TODO | Moderation + audit            |
| Web            | TODO | Employee/admin UI             |
| Testing        | TODO | Unit/integration/E2E          |
| Production     | TODO | CI/CD, monitoring, deployment |

## 30. Definition of Done for v1

Functional:

- ☐ Multi-tenant organizations
- ☐ Authentication and refresh tokens
- ☐ RBAC
- ☐ User profiles
- ☐ Departments
- ☐ Feed
- ☐ Posts/comments/reactions
- ☐ Direct/group/channel chat
- ☐ WebSocket realtime
- ☐ Presence
- ☐ Notifications
- ☐ File attachments
- ☐ Search
- ☐ Admin console
- ☐ Audit logs

Engineering:

- ☐ TypeScript
- ☐ PostgreSQL
- ☐ Redis
- ☐ Docker
- ☐ OpenAPI
- ☐ Validation
- ☐ Global error handling
- ☐ Rate limiting
- ☐ Security controls
- ☐ Tests
- ☐ CI/CD
- ☐ Health/readiness checks
- ☐ Structured logging

Product:

- ☐ Responsive UX
- ☐ Loading/empty/error states
- ☐ Permission-aware UI

- ☐ Realtime chat UX
- ☐ Onboarding
- ☐ Demo organization and accounts
- ☐ Production-quality README

## 31. Implementation Rules — Keep This Visible While Coding

1. Do not put business logic in Express route handlers.
2. Do not access another module's repository directly; use its application/domain contract.
3. Do not query tenant-owned data without organization scope.
4. Do not trust `organization_id` from the client when it can be derived from auth context.
5. Do not store raw passwords or raw refresh tokens.
6. Do not store large files in PostgreSQL.
7. Do not introduce microservices just for architecture aesthetics.
8. Prefer domain events for notifications and future SaaS integrations.
9. Keep API contracts backward-compatible within `/api/v1`.
10. Write tests for authorization and tenant isolation, not only happy paths.

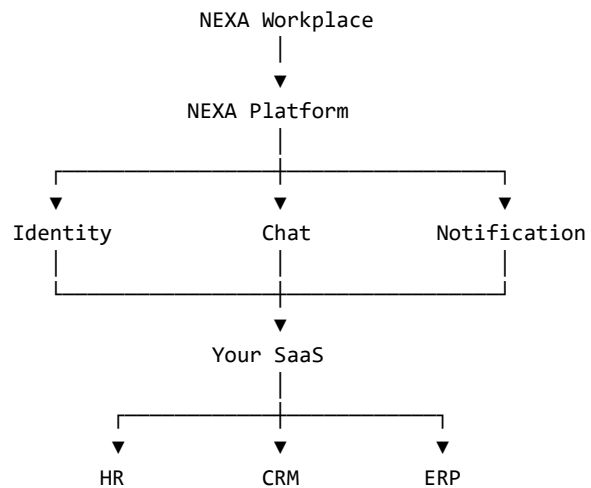
## 32. Suggested First Coding Session

1. Initialize monorepo
2. Create `apps/api` and `apps/web`
3. Configure TypeScript + lint + formatting
4. Add Docker Compose
5. Start PostgreSQL + Redis
6. Create database migration baseline
7. Create Express app
8. Implement config module
9. Implement request ID + logger
10. Implement global error handler
11. Add OpenAPI skeleton
12. Create Identity module skeleton
13. Commit: feat: initialize nexa platform foundation

## 33. Source Baseline

The original assignment/SRS is treated as the minimum functional baseline: Node.js + Express, JWT authentication, User/Post/Comment entities, post feed with pagination, post/comment CRUD, bcrypt password hashing, JWT middleware, owner authorization, validation and global error handling. NEXA v1 intentionally extends that baseline into a multi-tenant workplace product.

## 34. Final North Star



NEXA v1 is not merely a social-media assignment. It is the first productized communication layer for a future SaaS ecosystem. Build the boundaries carefully; add features only when they support that direction.